Презентация по лабораторной работе №2

Структуры данных Julia

Танрибергенов Э.

2026 г.

Российский университет дружбы народов, Москва, Россия

Информация

Докладчик

- Танрибергенов Эльдар
- студент 4 курса из группы НПИбд-01-22
- ФМиЕН, кафедра прикладной информатики и теории вероятностей
- Российский университет дружбы народов

Цели и задачи

Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

Задачи

- Изучить структуры данных в Julia
- Выполнить задания

Результаты

Кортежи

- Кортеж (Tuple) — структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы).

```
[1]: favoritelang = ("Python","Julia","R")

[1]: ("Python", "Julia", "R")
```

Рис. 1: Кортеж из элементов типа String

```
[2]: x1 = (1, 2, 3)

[2]: (1, 2, 3)
```

Рис. 2: Кортеж из целых чисел

```
[3]: x2 = (1, 2.0, "tmp")

[3]: (1, 2.0, "tmp")
```

Рис. 3: Кортеж из элементов разных типов

```
[4]: x3 = (a=2, b=1+2)

[4]: (a = 2, b = 3)
```

Рис. 4: Именованный кортеж

Примеры операций над кортежами

```
[5]: length(x2)

[5]: 3
```

Рис. 5: Операция length() над кортежом

```
[7]: c = x1[2] + x1[3]

[7]: 5
```

Рис. 6: Сложение элементов кортежа

```
[8]: x3.a, x3.b, x3[2]

[8]: (2, 3, 3)
```

Рис. 7: Обращение к элементам именованного кортежа

```
[9]: in("tmp", x2), 0 in x2

[9]: (true, false)
```

Рис. 8: Проверка вхождения элементов в кортеж

Словари

- Словарь — неупорядоченный набор связанных между собой по ключу данных.

```
[10]: phonebook = Dict("Иванов И.И." => ("867-5309","333-5544"), "Бухгалтерия" => "555-2368")

[10]: Dict{String, Any} with 2 entries:
        "Бухгалтерия" => "555-2368"
        "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 9: Создание словаря

```
[13]: pairs(phonebook)

[13]: Dict{String, Any} with 2 entries:
        "Бухгалтерия" => "555-2368"
        "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 10: Вывод заданных в словаре пар “ключ - значение”

```
[15]: phonebook["Сидоров П.С."] = "555-3344"

[15]: "555-3344"
```

Рис. 11: Добавление элемента в словарь

```
[17]: pop!(phonebook, "Иванов И.И.")

[17]: ("867-5309", "333-5544")
```

Рис. 12: Удаление ключа и связанных с ним значений из словаря

Множества

- Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа.

```
[19]: A = Set([1, 3, 4, 5])

[19]: Set{Int64} with 4 elements:
        5
        4
        3
        1
```

Рис. 13: Множество из 4 целочис. значений

```
[24]: S3 = Set([1,2,2,3,1,2,3,2,1]);
      S4 = Set([2,3,1]);
      issetequal(S3,S4)

[24]: true
```

Рис. 14: Проверка эквивалентности

```
[25]: C=union(S1,S2)

[25]: Set{Int64} with 4 elements:
        4
        2
        3
        1
```

Рис. 15: Объединение множеств

```
[26]: D = intersect(S1,S3)

[26]: Set{Int64} with 2 elements:
        2
        1
```

Рис. 16: Пересечение множеств

Массивы

- Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов.

```
[36]: a = [1, 2, 3]

[36]: 3-element Vector{Int64}:
       1
       2
       3
```

Рис. 17: Вектор-столбец

```
[40]: B = [[1 2 3]; [4 5 6]; [7 8 9]]

[40]: 3×3 Matrix{Int64}:
       1  2  3
       4  5  6
       7  8  9
```

Рис. 18: Многомерный массив (матрицы)

```
[44]: D = rand(4, 3, 2)

[44]: 4×3×2 Array{Float64, 3}:
      [:, :, 1] =
       0.165051  0.275015  0.674567
       0.405609  0.124098  0.695134
       0.518128  0.761084  0.342936
       0.177952  0.168147  0.61943

      [:, :, 2] =
       0.838742   0.46502    0.318713
       0.163425   0.839532   0.480772
       0.80575    0.375722   0.935615
       0.0732973  0.0185727  0.974309
```

Рис. 19: Трёхмерный массив

```
[45]: roots = [sqrt(i) for i in 1:10]

[45]: 10-element Vector{Float64}:
       1.0
       1.4142135623730951
       1.7320508075688772
       2.0
       2.23606797749979
       2.449489742783178
       2.6457513110645907
       2.8284271247461903
       3.0
       3.1622776601683795
```

Рис. 20: Массив из кв.корней цел.чисел 1-10

Примеры операций над массивами

```
[51]: fill(3.5,(3,2))

[51]: 3×2 Matrix{Float64}:
       3.5  3.5
       3.5  3.5
       3.5  3.5
```

Рис. 21: Заполнение массива 3x2 цифрами 3.5

```
[54]: a = collect(1:12)
      b = reshape(a,(2,6))

[54]: 2×6 Matrix{Int64}:
       1  3  5  7   9  11
       2  4  6  8  10  12
```

Рис. 22: Преобразование 1-мерного массива в двумерный

```
[63]: sort(ar,dims=1)

[63]: 10×5 Matrix{Int64}:
       12  12  10  11  10
       16  14  10  12  13
       18  15  12  13  14
       18  15  13  13  15
       18  15  13  15  16
       19  16  14  15  16
       19  16  15  16  17
       19  19  15  17  19
       20  19  17  18  20
       20  20  20  19  20
```

Рис. 23: Сортировка по столбцам

```
[65]: ar .> 14

[65]: 10×5 BitMatrix:
       1  1  0  1  0
       1  1  1  1  0
       1  0  1  0  0
       1  1  1  1  1
       1  1  0  1  1
       1  0  0  0  1
       1  1  0  1  1
       1  1  1  1  1
       0  1  0  0  1
       1  1  0  0  1
```

Рис. 24: Поэлементное сравнение с числом

Выполнение заданий

1) Даны множества: $A = \{0, 3, 4, 9\}$, $B = \{1, 3, 4, 7\}$, $C = \{0, 1, 2, 4, 7, 8, 9\}$. Найти $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$

```
[67]: A = Set([0, 3, 4, 9])
      B = Set([1, 3, 4, 7])
      C = Set([0, 1, 2, 4, 7, 8, 9])

      P = union(intersect(A,B), intersect(A,B), intersect(A,C), intersect(B,C))
```

```
[67]: Set{Int64} with 6 elements:
        0
        4
        7
        9
        3
        1
```

Рис. 25: Задание №1

Выполнение заданий

3.10) Создать вектор значений $y = e^x \cos(x)$ в точках $x = 3, 3.1, 3.2, \ldots, 6$, найдите среднее значение y;

[106]:
```
using Statistics
x = 3:0.1:6
y = exp.(x) .* cos.(x)
println("Вектор y: ", y)
mean_y = mean(y)
println("Среднее значение y: ", mean_y)
```

```
Вектор y: [-19.884530844146987, -22.178753389342127, -24.490696732801293, -26.77318244299338, -28.969237768093574, -31.0111864393
74516, -32.819774760338504, -34.30336011037369, -35.35719361853035, -35.86283371230767, -35.68773248011913, -34.68504225166807, -
32.693695428321746, -29.538816297262983, -25.032529229039966, -18.975233154958957, -11.157417389647478, -1.3620985182057503, 10.6
32038010191998, 25.046704998273004, 42.09920106253839, 61.99663027669454, 84.92906736250268, 111.0615860420258, 140.525075052787
5, 173.40577640857734, 209.73349424783467, 249.46844055885668, 292.4867067371223, 338.5643778585117, 387.36034029093076]
Среднее значение y: 53.11374594642971
```

Рис. 26: Задание №3.10

Выполнение заданий

5) Подключите пакет Primes. Сгенерируйте массив myprimes, в котором будут храниться первые 168 простых чисел. Определите 89-е наименьшее простое число. Получите срез массива с 89-го до 99-го элемента вкл., содержащий наименьшие простые числа.

```julia
using Primes

myprimes = primes(1000)[1:168]
eighty_ninth_prime = myprimes[89]
println("89-е наименьшее простое число: $eighty_ninth_prime")

slice_89_to_99 = myprimes[89:99]
println("Срез с 89-го по 99-й элемент: $slice_89_to_99")
```

```
89-е наименьшее простое число: 461
Срез с 89-го по 99-й элемент: [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]
```

Рис. 27: Задание №5

Выполнение заданий

6.1) $\sum_{i=10}^{100}(i^3 + 4i^2)$

```
[121]: sum_result = 0

       for i in 10:100
           sum_result += i^3 + 4 * i^2
       end

       println("Результат суммы: ", sum_result)

       Результат суммы: 26852735
```

Рис. 28: Задание №6.1

Выводы

Вывод

В результате выполнения лабораторной работы, я изучил несколько структур данных, реализованных в Julia, научился применять их и операции над ними для решения задач.